

FACULTAD DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

GOOGLE ANTIGRAVITY PARA PROGRAMACIÓN CIENTÍFICA

MSc. **Ignacio Tapia García**

Prof. Felipe Huerta

INTRODUCIÓN



01 Qué es Antigravity

02 Instalación y configuración

03 Modelos y límites

04 Buenos prompts

05 Agentes y multi-LLM

06 Ejemplo guiado

07 Tu proyecto

INTRODUCIÓN



Antigravity 2.0

App de escritorio

Centro de mando: proyectos, varios agentes en paralelo, tareas programadas. No incluye editor.



Antigravity IDE

Editor completo

El IDE original con panel de agente, artefactos y comprensión del código.



Antigravity CLI

Comando agy

Agente en la terminal. Ideal en servidores y clústeres por SSH.



Extensiones

Tu editor actual

VS Code, JetBrains, Visual Studio, Zed, Xcode y Eclipse.



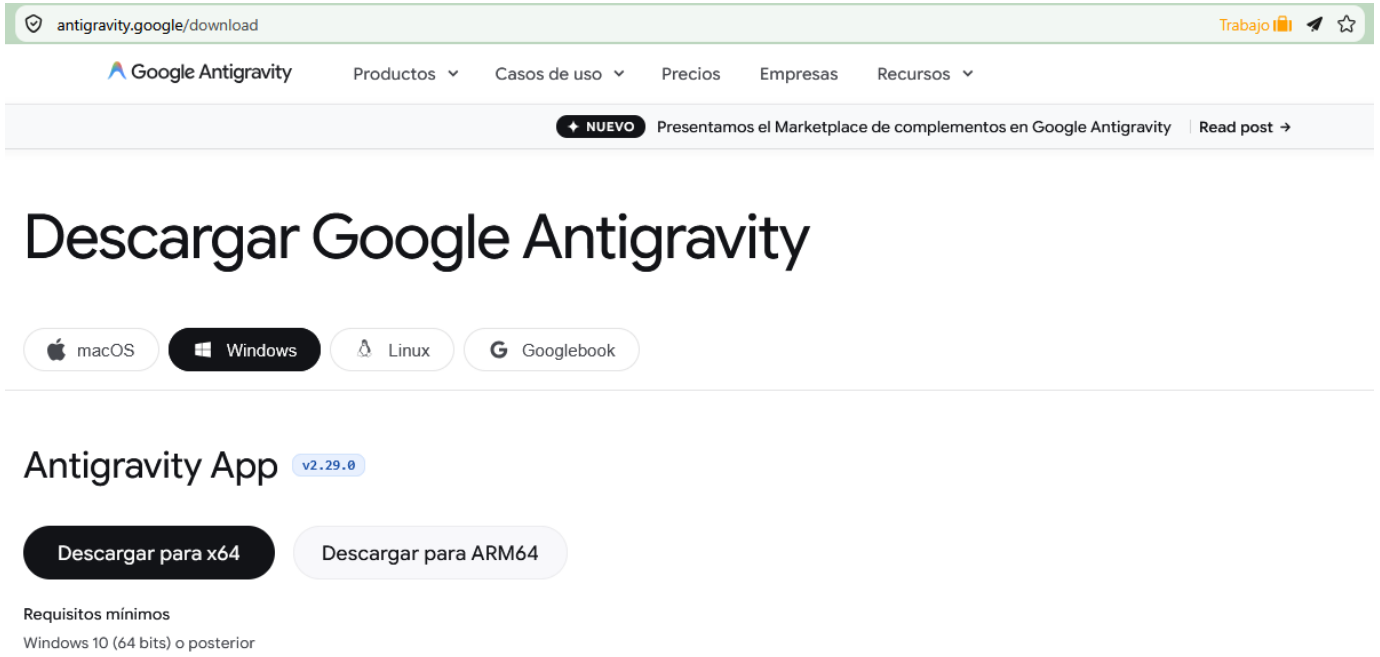
SDK Python

google-antigravity

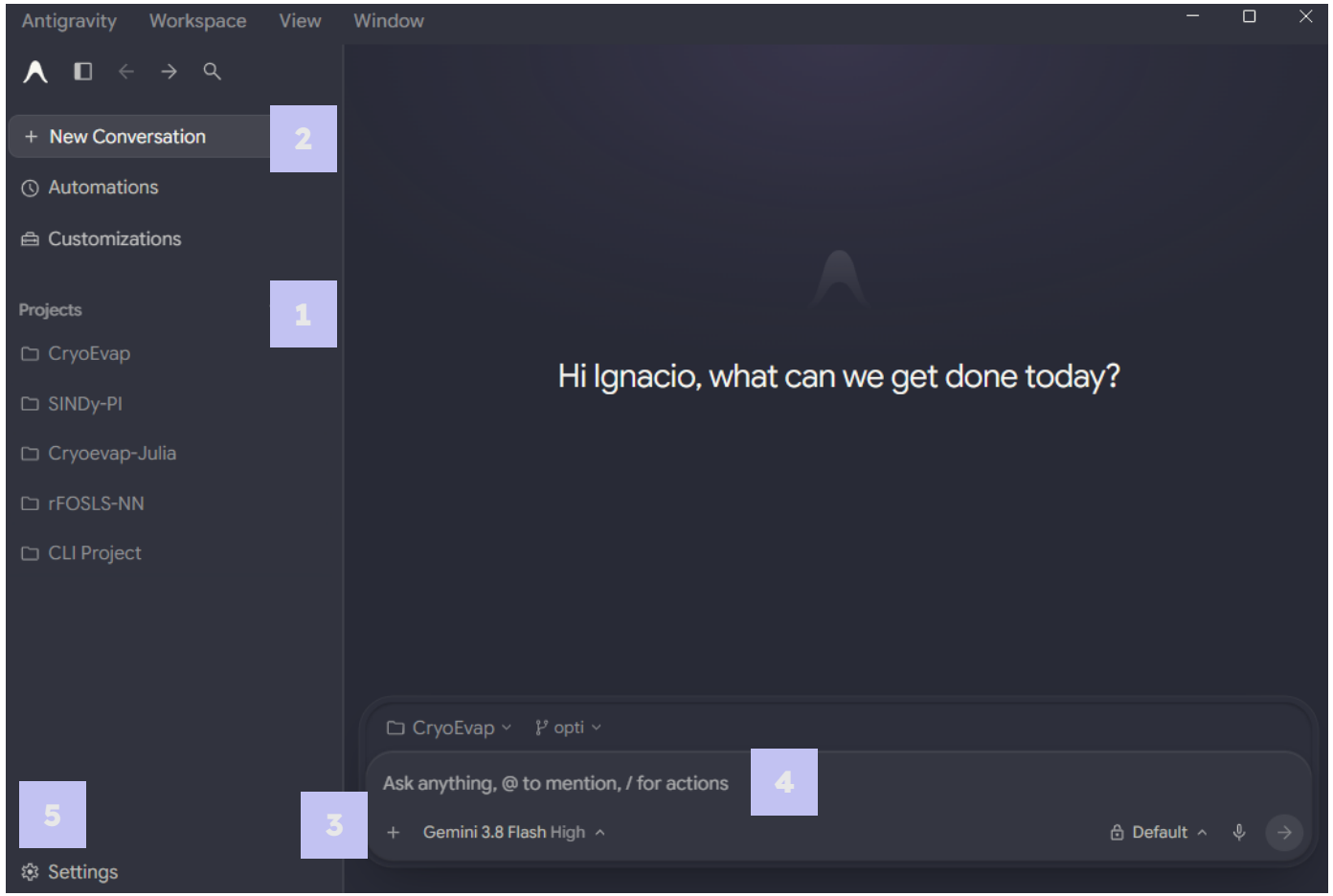
Construye agentes programáticos con herramientas, políticas y hooks.

INSTALACIÓN

Ingresar al link: **https://antigravity.google/download**



INTERFAZ



1 Proyectos

Carpetas + permisos que delimitan dónde actúa el agente.

2 Conversaciones

Hilos dentro de cada proyecto; historial y tareas programadas.

3 Selector de modelo

LLM y nivel de razonamiento (Low / Medium / High).

4 Caja de prompt

+ adjunta contexto, @ menciona archivos, / abre comandos.

5 Settings

Configuración global y personalizaciones.

INTERFAZ

¿Cómo crear un proyecto?

1. Crea una carpeta (puede tener código existente).
2. Projects → Create New Project → New Project y selecciona la carpeta.
3. Start first conversation: escribe tu primera instrucción.

Al lanzar el agente eliges el modo:



Local

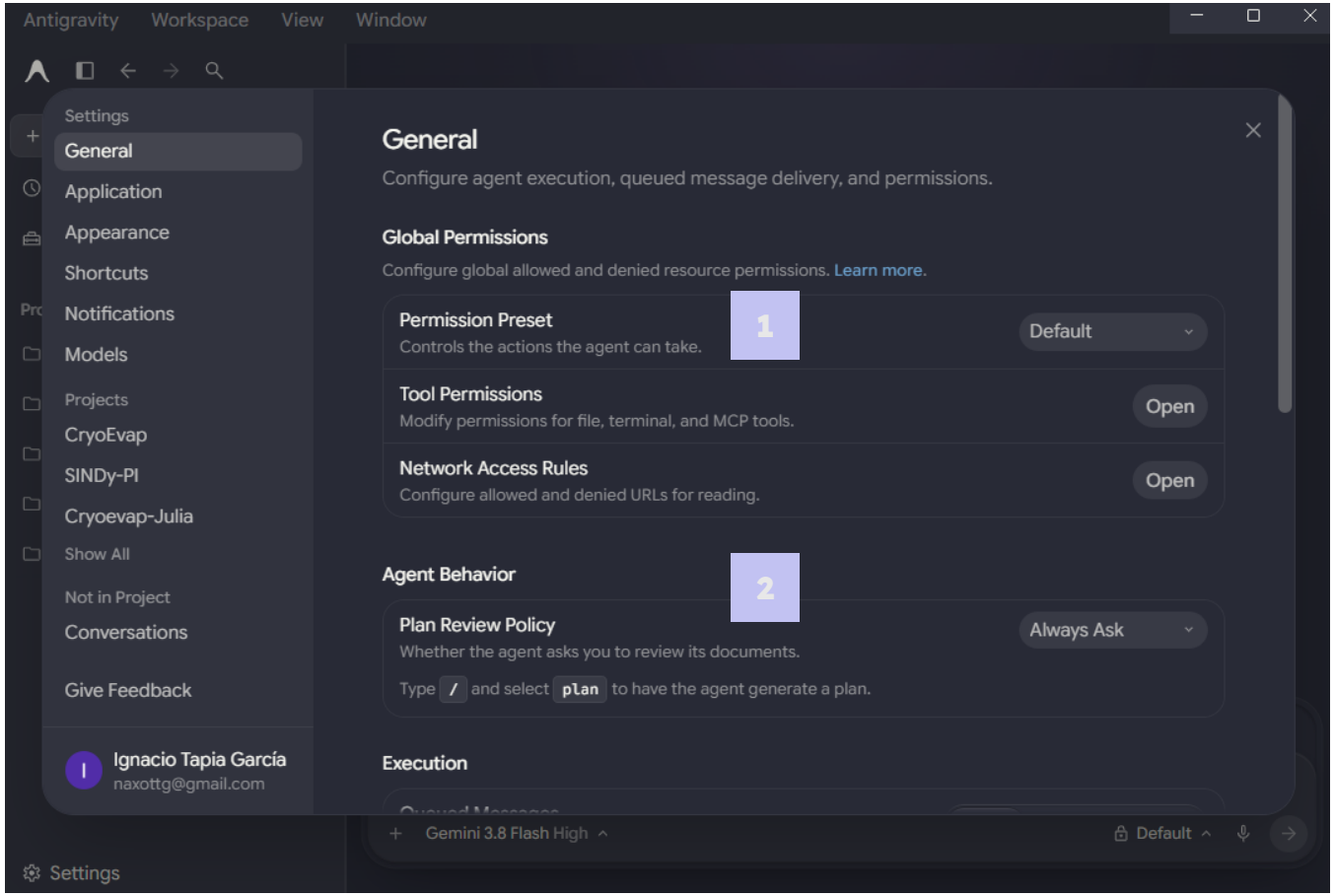
Trabaja directo sobre tus carpetas.
Rápido; ideal para scripts y exploración.



New worktree

Trabaja en un git worktree aislado.
Ideal para cambios grandes o varios agentes a la vez.

SETTINGS



1 **Permission Preset**

¿cuánta Libertad le damos al agente?

2 **Plan Review Policy**

¿Queremos aprobar lo que hará?

COMANDOS

Pensar antes de actuar

/grill-me	Te entrevista para aclarar requisitos
/plan	Plan de implementación que revisas y comentas
/goal	Trabaja de forma autónoma hasta terminar la tarea

Escalar y automatizar

/agents	Ver, dirigir y cambiar entre subagentes
/teamwork	Equipo de agentes para proyectos grandes (pago)
/boost	Razonamiento profundo multi-agente (Pro y Ultra)
/schedule	Tareas únicas o recurrentes
/browser	Prueba o investiga en Chrome

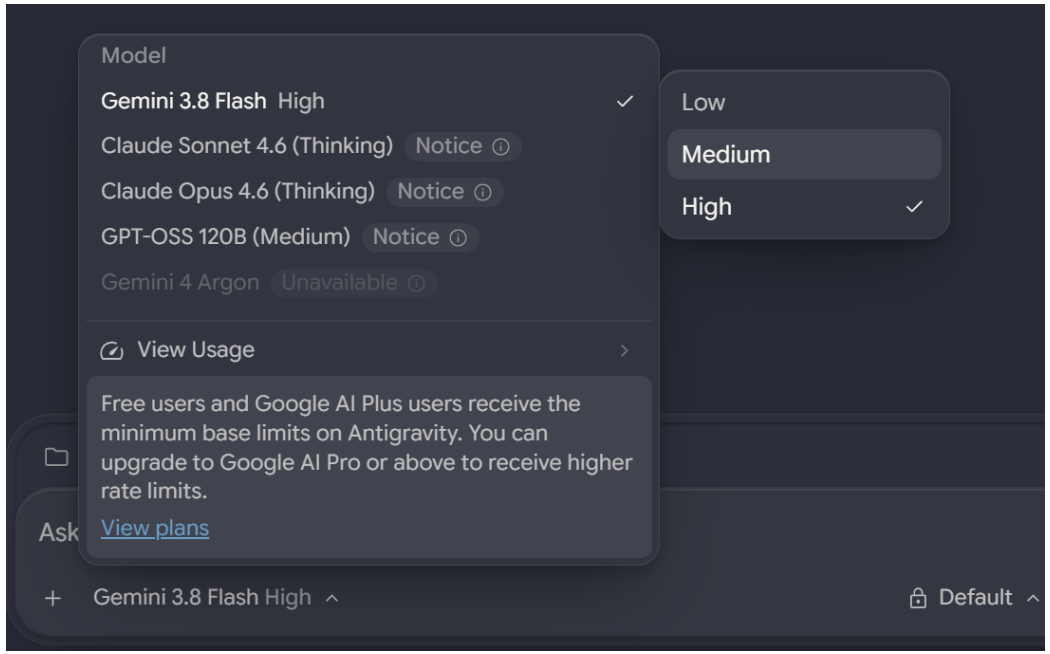
Revisar y controlar

/diff	Visor de cambios de cada turno
/usage	Consumo de cuota
/effort	Nivel de razonamiento
/resume	Retomar una conversación

Flujo recomendado para programación científica

/grill-me → **/plan** → **Comentar el plan** → **Proceed** → **/diff + tests** → **Revisor**

ELEGIR LLM



Modelo	Free / Plus	AI Pro	AI Ultra
Gemini 4 Argon	—	**Sí**	**Sí**
Gemini 3.8 Flash	**Sí**	**Sí**	**Sí**
Claude Sonnet / Opus 4.6*	**Sí**	**Sí**	—
GPT-OSS-120B*	**Sí**	**Sí**	**Sí**

*** Se retiran de Antigravity el 18 de noviembre de 2026.**

ELEGIR LLM

Tareas rápidas

Gemini 3.8 Flash · Low/Medium

Scripts de análisis, gráficos, tests, refactor pequeño, documentación.

Razonamiento profundo

Gemini 4 Argon u Opus 5.5 · High

Arquitectura de un código, depurar inestabilidades numéricas, portar algoritmos de un paper.

Explorar un repositorio

Subagente research con modelo flash

Mapear un código heredado o de un paper sin gastar el contexto principal.

Revisión cruzada

Otra familia de modelos

Si planificaste con Gemini, revisa con Claude (o al revés).

Regla práctica: empieza con Flash y sube de nivel solo si el agente se atasca. Planificar y revisar con el mismo modelo repite sus puntos ciegos.

¿CÓMO HACER UN BUEN PROMPT?

¿CÓMO HACER UN BUEN PROMPT?

Prompt vago

"mejora mi código de la ecuación de calor para que sea más preciso"

Qué suele pasar

- Cambia cosas que no pediste
- Elige el método por su cuenta
- No sabes si mejoró: no hay prueba
- Gasta más cuota iterando

Prompt estructurado

```
/plan
# Contexto
@src/heat1d.py resuelve u_t = α u_xx con diferencias finitas
explícitas (FTCS).

# Objetivo
Agregar Crank-Nicolson como method="cn".

#Restricciones
Solo NumPy/SciPy (sparse), float64, no cambies la firma de
solve().

# Verificación
Test con u = sin(πx)·exp(-απ²t); error L2 < 1e-4 con N=200;
mostrar orden 2 en un gráfico log-log (figs/convergencia.pdf).

# Entregable
Plan para revisar, luego diff + walkthrough.
```

¿CÓMO HACER UN BUEN PROMPT?

Planifica antes

Usa **/plan** y comenta el plan en vez de corregir el código después.

Deja que te pregunte

Con requisitos difusos, **/grill-me** hace que el agente te entreviste primero.

Contexto exacto

Menciona archivos con @ en lugar de describirlos.

Contexto persistente

Pon unidades, convenciones y entorno en AGENTS.md una sola vez.

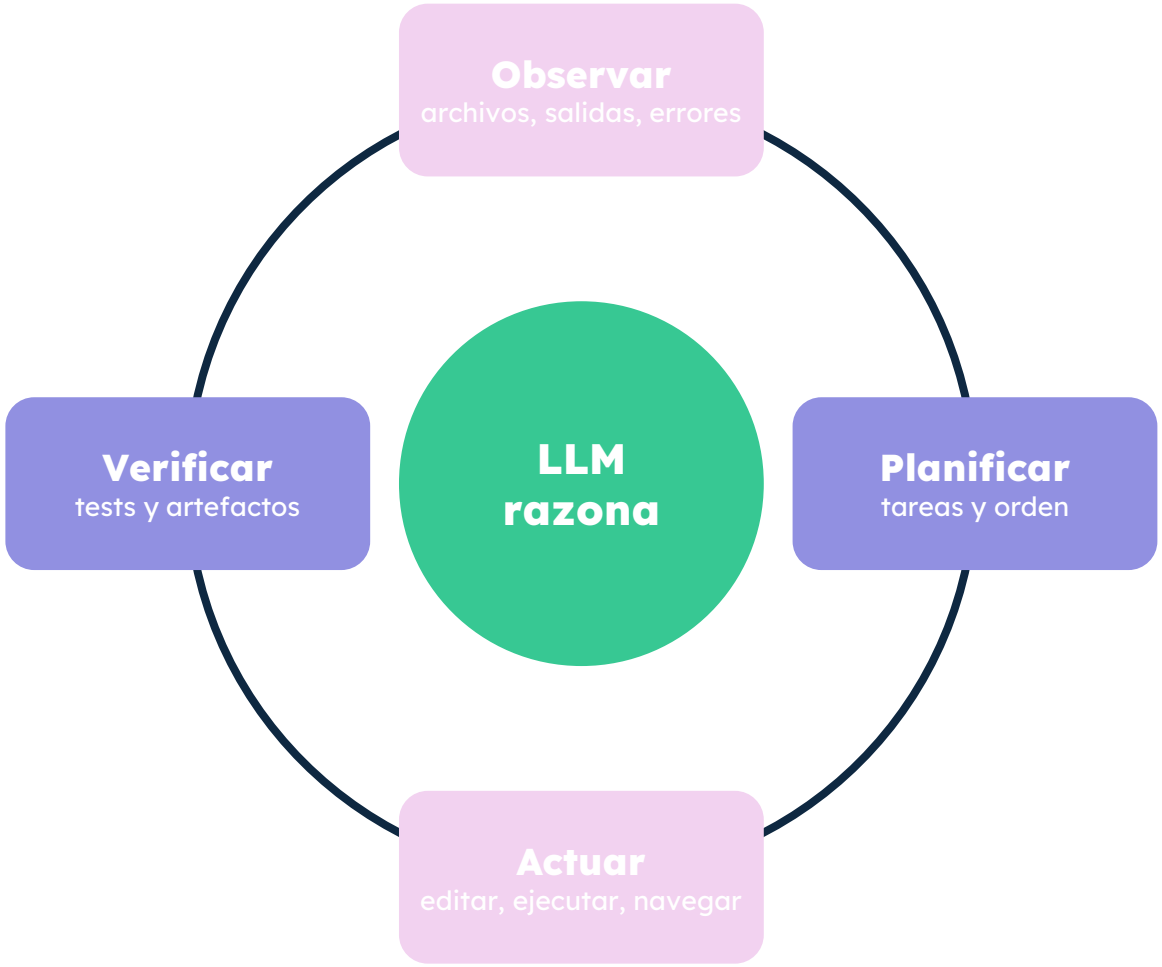
Éxito medible

Pide tests numéricos, tolerancias y figuras de comparación.

Tareas pequeñas

Una conversación por tema; divide lo grande en pasos verificables.

¿Qué es un agente?

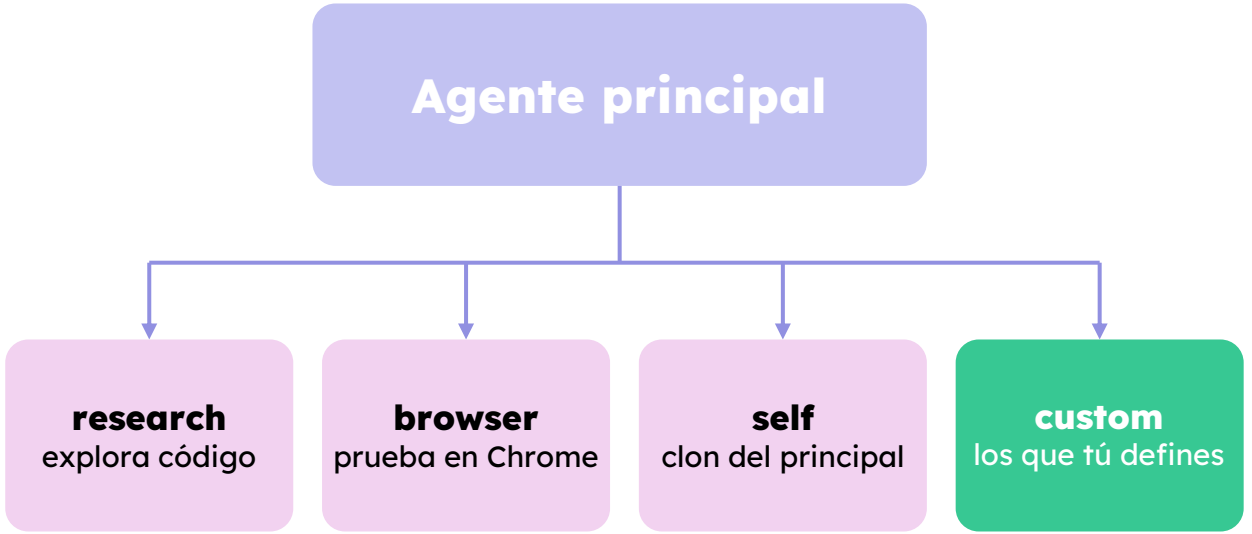


Agente =

LLM + herramientas + contexto/memoria + un objetivo + un bucle que se repite hasta cumplirlo.

	Chatbot	Agente
Acción	Responde texto	Edita y ejecuta
Pasos	Uno	Muchos, en bucle
Errores	Tú los detectas	Los ve y corrige
Rol humano	Copiar y pegar	Supervisar

Crear agentes con Markdowns



Crear agentes con Markdowns

```
# revisor-numerico.md
---
name: revisor-numerico
description: Revisa código científico: estabilidad,
  unidades, reproducibilidad y tests analíticos.
tools: [view_file, grep_search, run_command, replace_file_content]
subagent: true
mainAgent: false
model: pro              # flash | pro | Claude Opus | GPT 6

---

# System Prompt
Eres revisor de software científico. No edites archivos.

# Checklist
1. Estabilidad (CFL, condicionamiento, overflow).
2. Unidades y constantes físicas.
3. Semillas y reproducibilidad.
4. Tests vs solución analítica o caso límite.
Entrega hallazgos priorizados con archivo:línea y arreglo.
```

description

El orquestador la lee para decidir cuándo delegar. Escríbela como una regla de uso.

tools

Lista explícita de comandos que puede utilizar

model

Nivel de modelo: flash, pro, Argon, u otro.

Úsalo así: “Usa el subagente revisor-numerico para revisar @src/ y resume los 3 problemas más graves.”

Crear agentes con Markdowns

1 · Planificador

Gemini 4 Argon u Opus 5.5 (High)

Plan revisado por ti

→

2 · Implementadores

Subagentes model: flash en worktrees (branch)

Un módulo cada uno, en paralelo

→

3 · Revisor

Otra familia: p. ej. Claude Sonnet 5.5

LLM como juez del diff

↺ si el revisor encuentra problemas, vuelve al paso 2 con sus hallazgos

Cómo montarlo hoy en Antigravity

1. Conversación A con el modelo grande: **/plan** y aprueba el plan.
2. Pide al **orquestador** lanzar **subagentes flash** por módulo en modo branch.
3. Conversación B con un modelo de otra familia + revisor-numerico sobre el diff.
4. Lleva los hallazgos a A; repite hasta que pasen los tests.

Beneficios y contras de usar agentes

Beneficios

- Paralelismo: explorar, implementar y testear a la vez
- Modelos diversos reducen puntos ciegos
- Nivel gratuito para empezar a probar
- Automatiza lo repetitivo con /schedule
- Reglas, skills y agentes se comparten en el grupo

Contras y riesgos

- Cuota no publicada; multi-agente la consume rápido
- Errores numéricos sutiles que parecen correctos
- Conflictos al fusionar trabajo de varios agentes
- Tu código y datos se procesan en la nube
- Revisar diffs exige tiempo y criterio experto

Lotka-Volterra con diferencias finitas

El modelo

$$\frac{dx}{dt} = \alpha x - \beta xy$$

x: presas y: depredadores

Euler explícito (diferencias hacia adelante)

$$x_{n+1} = x_n + \Delta t\,(\alpha x_n - \beta x_n y_n)$$

1. Implementar

Euler explícito en un bucle; parámetros en un dataclass.

2. Precisión

Caso con solución exacta y refinamiento de Δt.

3. Coherencia física

Positividad, equilibrio, cantidad conservada.

4. Condiciones

Barrido de α, β y condiciones iniciales.

5. Reporte

Lotka-Volterra con diferencias finitas

Sub agentes

```
# Graficador.md
Subagente que ejecuta scripts para generar graficos y escribir
reporte de resultados
```

```
# revisor_numerico.md
Subagente que revisa implementación y convergencia numérica.
```

```
# tester_fisico.md
Subagente que revisa la física y consistencia del modelo.
```

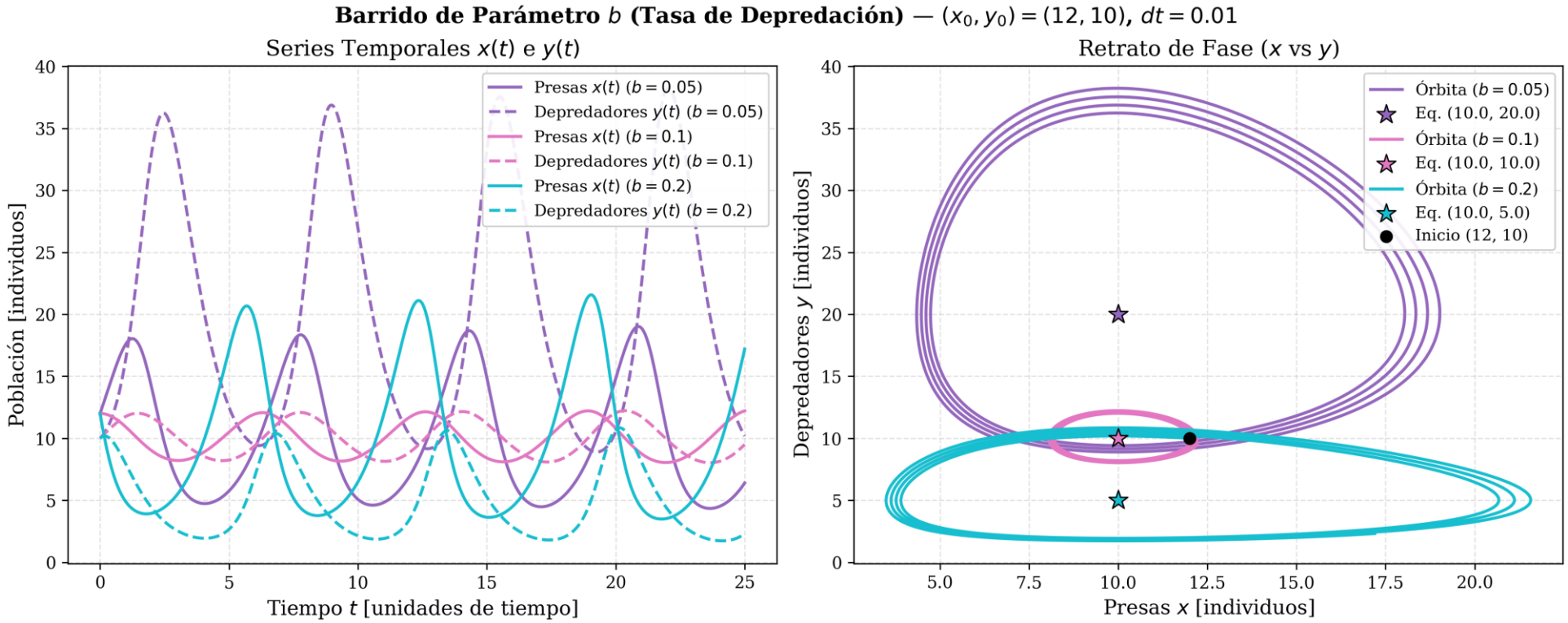
Agente principal

```
# orquestador.md

Agente principal que crea el
modelo, llama a los
subagentes y responde en el
chat.

Se sugiere invocar a este
agente directamente en el
chat, copiando y pegando el
texto escrito en el archivo
orquestador.md
```

Lotka-Volterra con diferencias finitas



¿Tiene sentido? Recuerda revisar los resultados que entrega.

Proyecto del curso

Evaporación

Base: Compuesto puro
Nueva feature: Mezcla multicomponente: equilibrio con Raoult, difusión y balance por especie
Límite: Con un solo componente (x = 1) reproduce el caso puro

Ecuación de estado

Base: Gas ideal
Nueva feature: Peng–Robinson: cúbica en Z con parámetros a, b y ω
Límite: A baja presión $Z \to 1$ y coincide con gas ideal

Estanque

Base: Solo almacenamiento
Nueva feature: Carga y descarga: caudales de entrada y salida en el balance
Límite: Con caudales nulos reproduce el almacenamiento

[Tu proyecto]

Base: [Modelo actual]
Nueva feature: [Ecuaciones o términos nuevos]
Límite: [Condición que apaga la feature]

Regla de oro: con la feature apagada, el código nuevo debe reproducir el código base: mismos resultados y mismos tests.

Proyecto del curso

Completa estos datos

Código base

[Repositorio o ruta; cómo se ejecuta]

Modelo actual

[PDE, condiciones de borde, método numérico]

Nueva feature

[Ecuaciones o términos que se agregan]

Límite de consistencia

[Condición en que se recupera el modelo base]

Criterio de éxito

[Tests y tolerancias que deben pasar]

1. el código base original queda intacto
2. subagente research: dónde vive cada término de la PDE
3. correr casos de referencia y guardarlos como tests de regresión
4. /grill-me + /plan con ecuaciones nuevas y límite de consistencia
5. propiedades → términos de la PDE → bordes, con tests en cada fase
6. física, barrido de condiciones, revisión cruzada y reporte

Proyecto del curso

Prompt tipo para el agente principal

```
/grill-me

Contexto: @src/ es un simulador funcional de [fenómeno]
  que resuelve [PDE actual] con [método numérico].

Objetivo: agregar [feature] mediante [ecuaciones nuevas],
  activable con un parámetro (p. ej. eos="pr").

Restricciones: no cambiar la API ni borrar tests;
  float64; unidades SI.

Verificación:
  1. Con [feature apagada] = línea base (rtol 1e-8).
  2. Física: [conservación, cotas, signos esperados].
  3. Convergencia de malla con la feature activa.

Entregable: plan → código → tests → figs/ con
  [condiciones a comparar] → reports/reporte.md
```

Crear sub agentes para tareas específicas paralelizables:

- Búsqueda web.
- Verificar implementación.
- Verificación física.
- Ejecución de códigos y escritura de reporte.

Escribir un archivo .md por subagente!



FACULTAD DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

GOOGLE ANTIGRAVITY PARA PROGRAMACIÓN CIENTÍFICA

MSc. **Ignacio Tapia García**

Prof. Felipe Huerta